جامعة مولاي إسماعيل
UNIVERSITÉ MOULAY ISMAÏL

ENSAM
المدرسة الوطنية العليا للفنون و المهن
ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS

LINUX ADMINISTRATION

Réalisé par : KAICHE Zayd

Encadré par : Prof. Abdeslam Ahmadi

Date : 18/10/2026

Filière : Génie Informatique - Ingénierie Logicielle et Systèmes Intelligents

Table of contents

Introduction : 5
1. Key concepts and prerequisites 6
2. Observing the host machine 7
2.1 Checking the host's resources 7
2.2 Checking hardware virtualization 7
3. Creating the virtual machine 8
3.1 Creating the VM 8
3.2 Setting the memory and processors 8
3.3 Creating the virtual disk 9
3.4 Checking the settings before starting 9
4. Installing Ubuntu Server 11
4.1 Booting from ISO 11
4.2 Language and keyboard layout 11
4.3 Network configuration 12
4.4 Storage configuration 13
4.5 User and server identity 14
4.6 Skipping Ubuntu Pro 14
4.7 Installing OpenSSH 15
4.8 Rebooting and removing the ISO 16
5. First steps in Ubuntu and network modes 19
5.1 Loggin in 19
5.2 Changing the console font (optional) 19
5.3 Checking the system 20
5.3.1 Server name and version 20
5.3.2 Memory and processor 20
5.3.3 Disk and file systems 20
5.3.4 Summary of the commands : 21
5.4 Checking the network 22
5.4.1 IP address and routing 22

5.4.2 Connectivity test 22
5.5 Checking SSH and SSHD services 23
5.6 VirtualBox network modes 24
5.6.1 Solution 1: Bridged Adapter 24
5.6.2 Solution 2: Port forwarding (staying in NAT) 26
5.6.3 Conclusion 27
6. Snapshots and VBoxManage 28
6.1 Shutting down the virtual machine cleanly 28
6.2 Creating the reference snapshot 28
6.3 Snapshot or Backup 30
6.4 Discovering VBoxManage 30
6.4.1 Adding VBoxManage to the PATH 30
6.2 Listing the virtual machines 31
6.2 Showing the details of a machine 31
7. Cloning the virtual machine 33
7.1 Creating the clone 33
7.2 Checking the clone 34
Conclusion 35

Table of figures

Figure 1: VirtualBox "New" window with the name, folder, ISO, type, and version filled in 8
Figure 2: the "Hardware" step showing 2048 MB of RAM and 2 processors 9
Figure 3: the "Hard disk" step with 60 GB and Pre-Allocate Full Size unchecked 9
Figure 4: Details of the virtual machine 10
Figure 5: the boot menu of the installer with "Try or Install Ubuntu Server" 11
Figure 6: the keyboard configuration screen with Layout and Variant selected 12
Figure 7: the network screen showing the interface with a DHCP address 13
Figure 8: the storage screen showing the 60 GB virtual disk selected 13
Figure 9: the profile setup screen filled in 14
Figure 10: the "Upgrade to Ubuntu Pro" screen with "Skip for now" selected 15
Figure 11: the SSH setup screen with "Install OpenSSH server" checked 16
Figure 12: the "Installation complete" screen with "Reboot Now" 17
Figure 13: the login screen and the welcome message after a successful login 19
Figure 14: the console after updating the font, showing the larger text 19
Figure 15: output of hostname and cat /etc/os-release | grep VERSION 20
Figure 16: output of free -h and lscpu | grep "CPU(s):" showing 1.6 GiB of memory and 2 CPUs 20
Figure 17: output of lsblk 21
Figure 18: output of df -h 21
Figure 19: output of ip addr and ip route, showing the interface, its IP address, and the default route 22
Figure 20: output of two ping commands with 0% packet loss 23
Figure 21: output of systemctl status ssh showing "active (running)" and "enabled" 23
Figure 22: Settings -> Network with "Bridged Adapter" selected 24
Figure 23: the output of ip a showing the new IP address 25
Figure 24: the SSH connection from the Windows command prompt 25
Figure 25: the port forwarding rule in VirtualBox 26
Figure 26: the SSH connection using port 2222 26
Figure 27: the VM showing "Powered Off" in VirtualBox Manager 28
Figure 28: the "Take Snapshot" window with the name Clean-Install 29
Figure 29: the Snapshots tab showing Clean-Install and Current State 29
Figure 30: the Environment Variables window with the VirtualBox folder added to Path 31
Figure 31: output of VBoxManage list vms in PowerShell 31
Figure 32: output of VBoxManage showvminfo, filtered on memory, CPUs, and network 32
Figure 33: the Clone Virtual Machine window with these options selected 33
Figure 34: the VirtualBox Manager showing both machines, Ubuntu Server and Ubuntu Client #1 34

Introduction :

For years, Linux distributions have stood out as powerful operating systems that give users full control over their systems while consuming very little memory and CPU. This efficiency, combined with their stability, security, and flexibility, is the reason Linux is the most widely used operating system in server hosting and data centers around the world. Because so much of the modern digital infrastructure runs on Linux, learning its administration has become an essential skill for anyone working in IT. Mastering tasks such as managing users and permissions, configuring services and networks, automating processes with scripts, monitoring performance, and securing a system allows professionals to deploy, maintain and troubleshoot applications reliably.

In this session, we will take our first practical steps by learning how to set up Ubuntu Server on a virtual machine with Oracle VirtualBox, a safe and flexible environment for practicing without risking a real system. We will install and check the server, connect to it remotely through SSH using NAT and Bridged network modes, and protect our work with snapshots and clones. For future software and systems engineers in particular, these skills provide the foundation needed to work with technologies like Docker, cloud services, and DevOps pipelines, while also opening the door to many career opportunities in a field where demand for Linux administration continues to grow.

1. Key concepts and prerequisites

Before building anything, it is important to understand three roles.

- The **host machine** is the physical computer, in our case a Windows PC.
- The **hypervisor** is the software that creates and manages virtual machines, here Oracle VirtualBox.
- The **guest machine** is the virtual machine running inside it, here Ubuntu Server 26.04.1 LTS.

The hypervisor shares the host's physical resources (CPU, RAM, and storage) with each guest, so they must be allocated carefully.

To follow this session, you will need:

- Windows 10 or 11, 64-bit, with at least 16 GB of RAM and 8 CPU cores.
- About 50 GB of free disk space
- Hardware virtualization enabled in the BIOS/UEFI (this is the most common reason a virtual machine fails to start)
- Oracle VirtualBox installed
- The Ubuntu Server 26.04.1 LTS ISO image

Installing VirtualBox and downloading Ubuntu Server

1. Go to virtualbox.org, open the Download page, and download the installer for **Windows hosts**.
2. Run the installer and keep the default options.
3. Go to Ubuntu Server Download Page and download **Ubuntu Server 26.04.1 LTS** ISO image.
4. Save the ISO in a folder that is easy to find, since you will select it when creating the virtual machine.

2. Observing the host machine

2.1 Checking the host's resources

Before creating a virtual machine, we need to know what the host can offer, because every virtual machine borrows its resources. On Windows, press **Ctrl + Shift + Esc** to open Task Manager (click **More details** if only a small window appears), then open the **Performance** tab. The **CPU** panel shows the processor model, the number of cores, and the logical processors. The **Memory** panel shows the installed RAM, and the **Disk** panel shows the storage.

The machine used in this article has the following characteristics:

1. Processor : **Intel Core i9-13900H**
2. Cores : **14**
3. Logical processors : **20**
4. RAM : **64 GB DDR5**
5. Storage : **1 TB NVMe SSD (101 GB free on C:)**
6. Graphics card : **NVIDIA GeForce RTX 4070 (8 GB VRAM)**
7. Virtualization : **Enabled**

2.2 Checking hardware virtualization

A hypervisor needs the processor's virtualization support to run 64-bit virtual machines efficiently. In Task Manager, go to **Performance** then **CPU** and look at the **Virtualization** line at the bottom right. It must show **Enabled**. On our machine it does.

If it shows **Disabled**, you need to turn it on in the BIOS/UEFI:

1. Restart the computer and press the BIOS key during startup. It depends on the manufacturer, commonly **F2**, **Del**, **F10**, or **ESC**.

2. Look for the virtualization option, usually under Advanced, CPU Configuration, or Security. On Intel processors it is called **Intel Virtualization Technology (VT-x)**, and on AMD processors **SVM Mode**.

3. Set it to **Enabled**.

4. Save and exit (usually **F10**), then let Windows restart.

5. Open Task Manager again to confirm it now shows **Enabled**.

3. Creating the virtual machine

With the host checked, we can create our virtual machine in VirtualBox. We will build it in four stages: the basic settings, the memory and processors, the virtual disk, and a final check.

3.1 Creating the VM

Open VirtualBox and click **New**. In the wizard, enter the following settings:

- **Name** : Ubuntu Server
- **ISO Image :** Select the Ubuntu Server 26.04.1 LTS ISO downloaded earlier.
- **Type** : Linux
- **Version** : Ubuntu (64-bit)



Figure 1: VirtualBox "New" window with the name, folder, ISO, type, and version filled in

3.2 Setting the memory and processors

In the hardware section of the wizard, set:

- **Base memory:** 2048 MB
- **Processors** : 2

Ubuntu Server has no heavy graphical interface, so 2 GB of RAM is enough, and it leaves plenty of memory for Windows and for the virtual machines we will create later.

A common question is why we don't give the virtual machine all the processors of the host. The reason is that the host itself and every other running virtual machine need CPU time too. If the total number of virtual processors greatly exceeds what the physical processor can provide, the machines compete for it and everything slows down. Our host has 20 logical processors, so it could give more, but 2 vCPUs are enough for this lab and keep the same configuration for readers with more modest PCs.

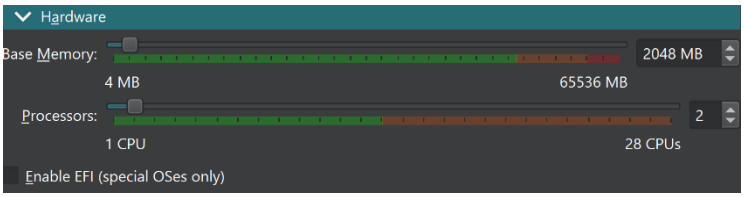


Figure 2: the "Hardware" step showing 2048 MB of RAM and 2 processors

3.3 Creating the virtual disk

In the hard disk section, choose to create a new hard disk and set:

- **Size** : 60 GB
- **Format** : VDI (VirtualBox Disk Image). This is the default format. In the wizard it is only shown in Expert Mode.
- **Allocation** : Dynamically allocated, which means leaving the **Pre-Allocate Full Size** box unchecked.

The 60 GB is the maximum capacity of the disk. With dynamic allocation, the file on the Windows drive starts small and grows only as data is written to it, so it does not take 60 GB immediately.

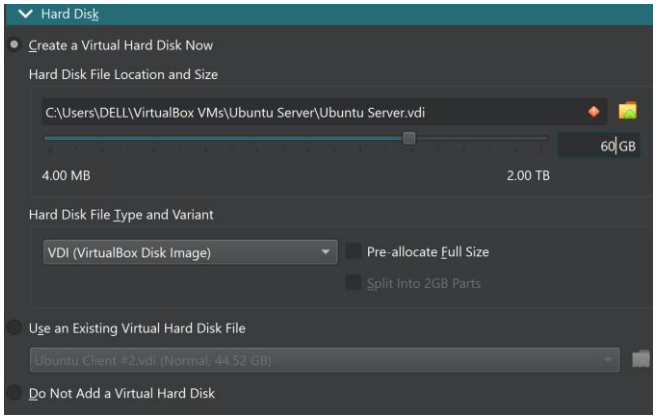


Figure 3: the "Hard disk" step with 60 GB and Pre-Allocate Full Size unchecked

3.4 Checking the settings before starting

Before starting the machine, select it and open **Settings** to verify that everything is correct:

- **System (Motherboard)** : 2048 MB of RAM
- **System (Processor)** : 2 CPUs
- **Storage** : 60 GB virtual disk and the Ubuntu Server ISO in the optical drive.
- **Network (Adapter 1)** : enabled and attached to **NAT**.

NAT is the default network mode. It lets the virtual machine reach the internet through the host, and we will come back to it in the section on network modes.

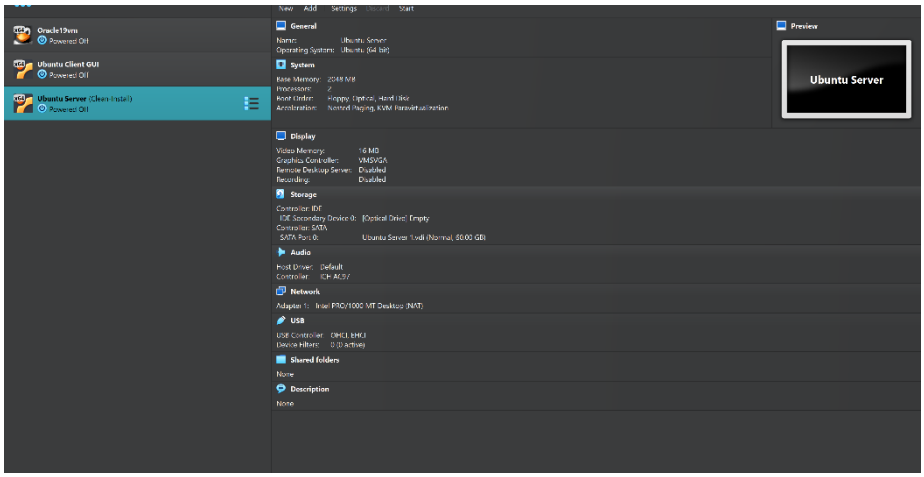


Figure 4: Details of the virtual machine

Summary of the configuration :

- **VM name** : Ubuntu Server
- **RAM** : 2 GB
- **vCPU** : 2
- **System disk** : 60 GB, VDI, dynamically allocated.
- **Network** : NAT
- **OS** : Ubuntu Server 26.04.1 LTS, 64-bit

References :

[1] A. Ahmadi, TP 1: Mise en place de l'environnement de virtualisation, Cloud Computing et Virtualisation (Partie B), ENSAM Meknès.

[2] Oracle, Oracle VirtualBox User Manual

4. Installing Ubuntu Server

Now that the virtual machine is ready, we can install Ubuntu Server 26.04.1 LTS. The installer is text-based, so the mouse does not work inside it. We navigate with the **Tab** and arrow keys and validate with **Enter**.

4.1 Booting from ISO

Select the virtual machine and click **Start**. The Ubuntu Server installer appears. In the first menu, choose **Try or Install Ubuntu Server** and press **Enter**.



Figure 5: the boot menu of the installer with "Try or Install Ubuntu Server"

4.2 Language and keyboard layout

Choose the language of the installer. For IT students, English is recommended, since most documentation and error messages are in English. If the installer offers an update at this point, continue without updating. Then, on the keyboard screen, set the **Layout** and the **Variant** to match your physical keyboard. For an **AZERTY** keyboard, choose French for both. Use **Tab** to move between fields, then select **Done** and press **Enter**.

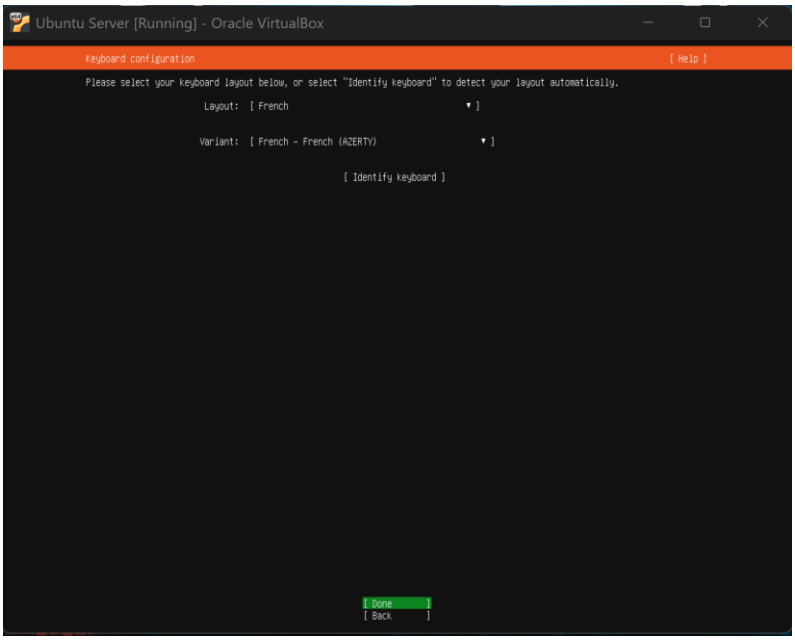


Figure 6: the keyboard configuration screen with Layout and Variant selected

4.3 Network configuration

Leave the network interface in **DHCP** for now. Because the virtual machine uses NAT, VirtualBox provides an IP address and internet access automatically. The interface name may be something like **enp0s3**, and the exact name can vary. Then:

- **Proxy :** Leave it empty and select **Done**.
- **Mirror :** Keep the default Ubuntu mirror address and select **Done**.

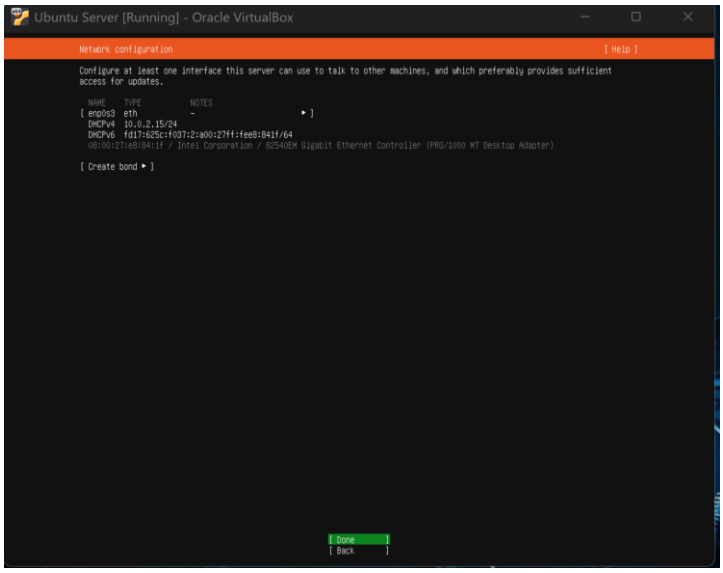


Figure 7: the network screen showing the interface with a DHCP address

4.4 Storage configuration

Choose **Use an entire disk**, and check carefully that the selected disk is the 60 GB virtual disk, not a physical disk of the host. Select **Done** and press **Enter**.

On the summary screen, select **Done** again, then confirm with **Continue** when the installer warns that the disk will be formatted.

This is safe because the installer only sees the virtual disk created for this VM, and Windows is not affected.

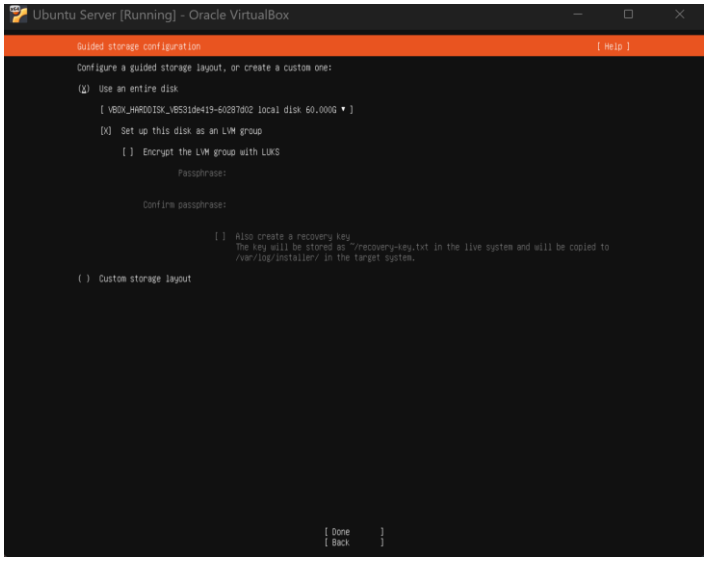


Figure 8: the storage screen showing the 60 GB virtual disk selected

4.5 User and server identity

Fill the profile screen:

- **Your name** : any name you like
- **Your server's name :** a name without spaces, for example **ubuntu-server**
- **Username** : for example **zaayd**
- **Password** : choose your own password and remember it, since you will need it to login and to use **sudo**.

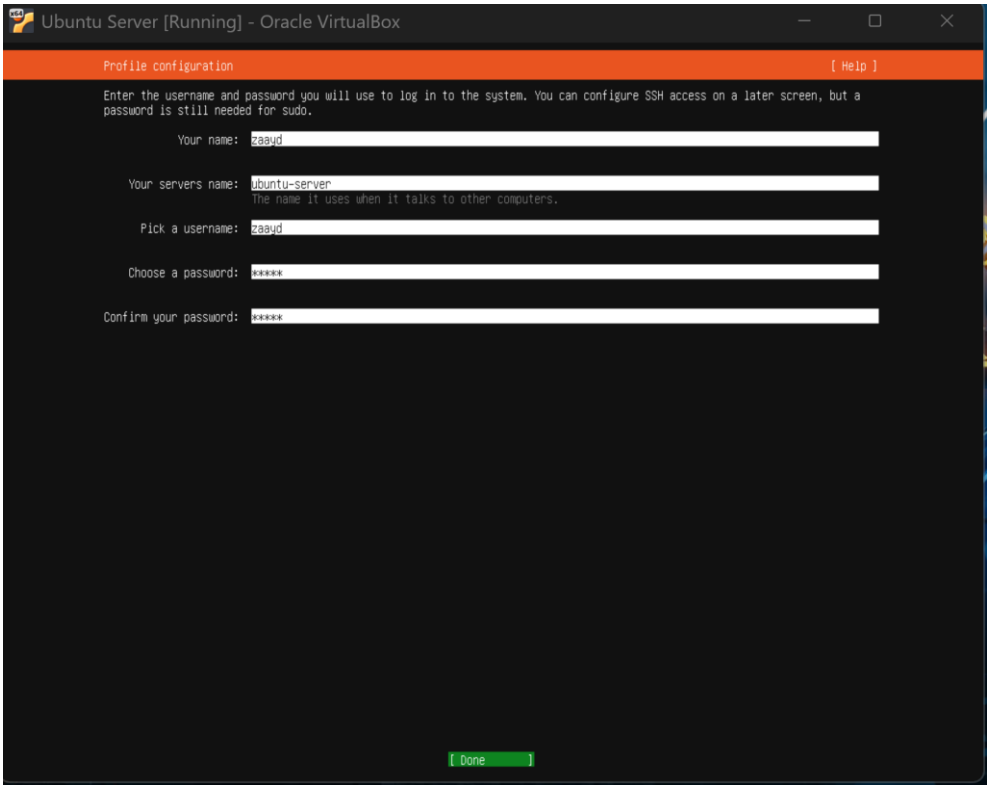


Figure 9: the profile setup screen filled in

4.6 Skipping Ubuntu Pro

The installer offers an upgrade to Ubuntu Pro, a paid support subscription. It is not needed for this lab, so select **Skip for now**.

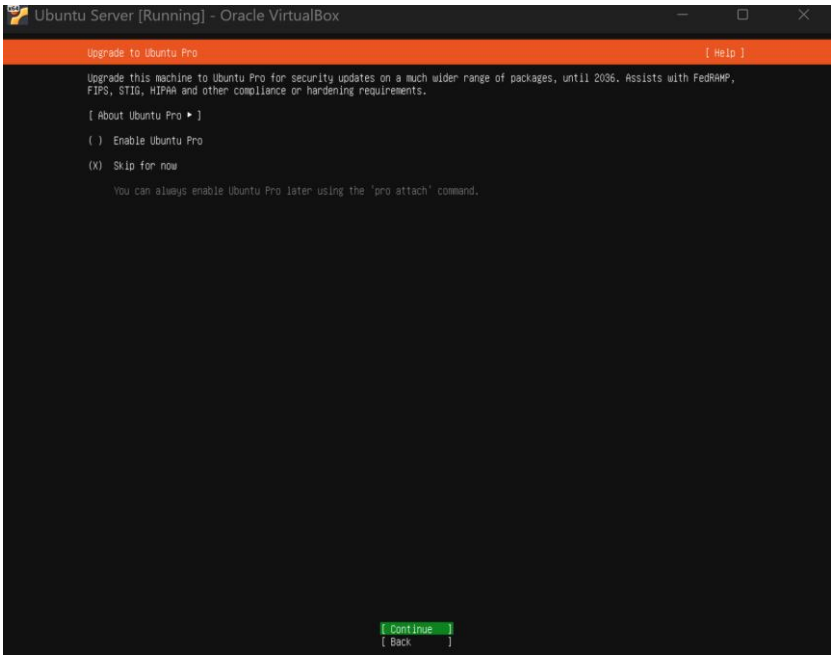


Figure 10: the "Upgrade to Ubuntu Pro" screen with "Skip for now" selected

4.7 Installing OpenSSH

When the installer proposes the SSH server, select **Install OpenSSH server**. Press **Enter** on the option until an **X** appears in the brackets, then select **Done**. SSH lets us administer the server remotely from Windows, which we will use later in this article.

On the next screen, **Featured server snaps**, do not select any package. Just select **Done**.

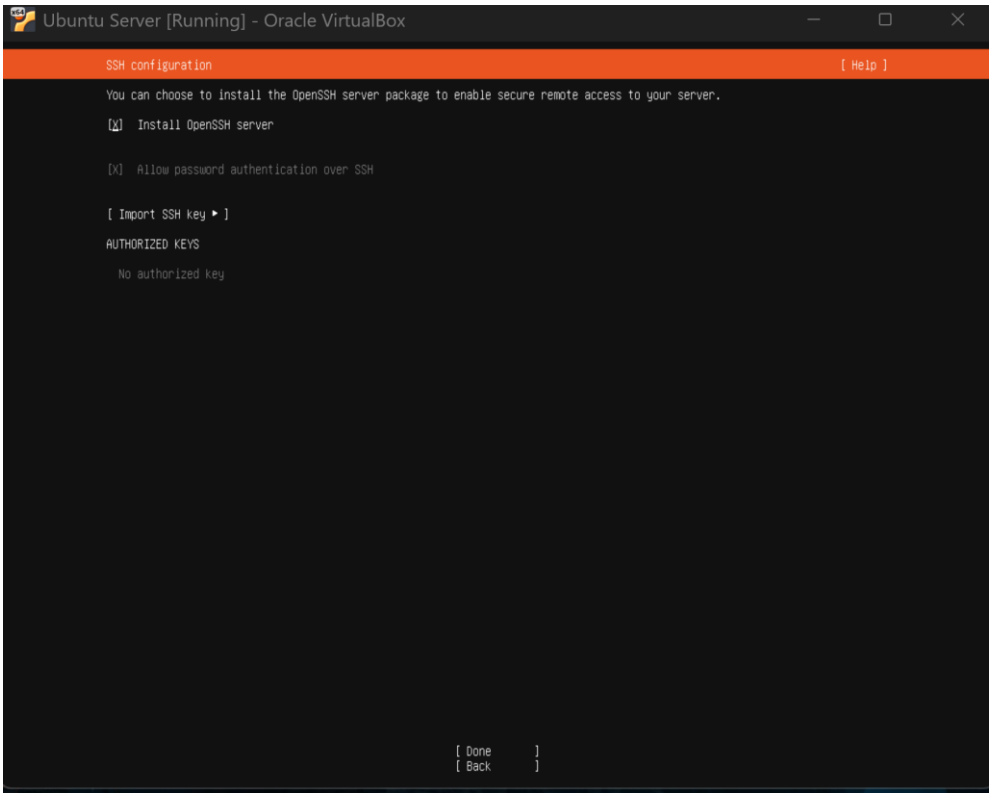


Figure 11: the SSH setup screen with "Install OpenSSH server" checked

4.8 Rebooting and removing the ISO

Wait for the installation to finish, then select **Reboot Now**. If the installer asks you to remove the installation medium, in the VirtualBox menu go to **Devices -> Optical Drives -> Remove disk from virtual drive**, then press **Enter**. Removing the ISO prevents the virtual machine from booting into the installer again.

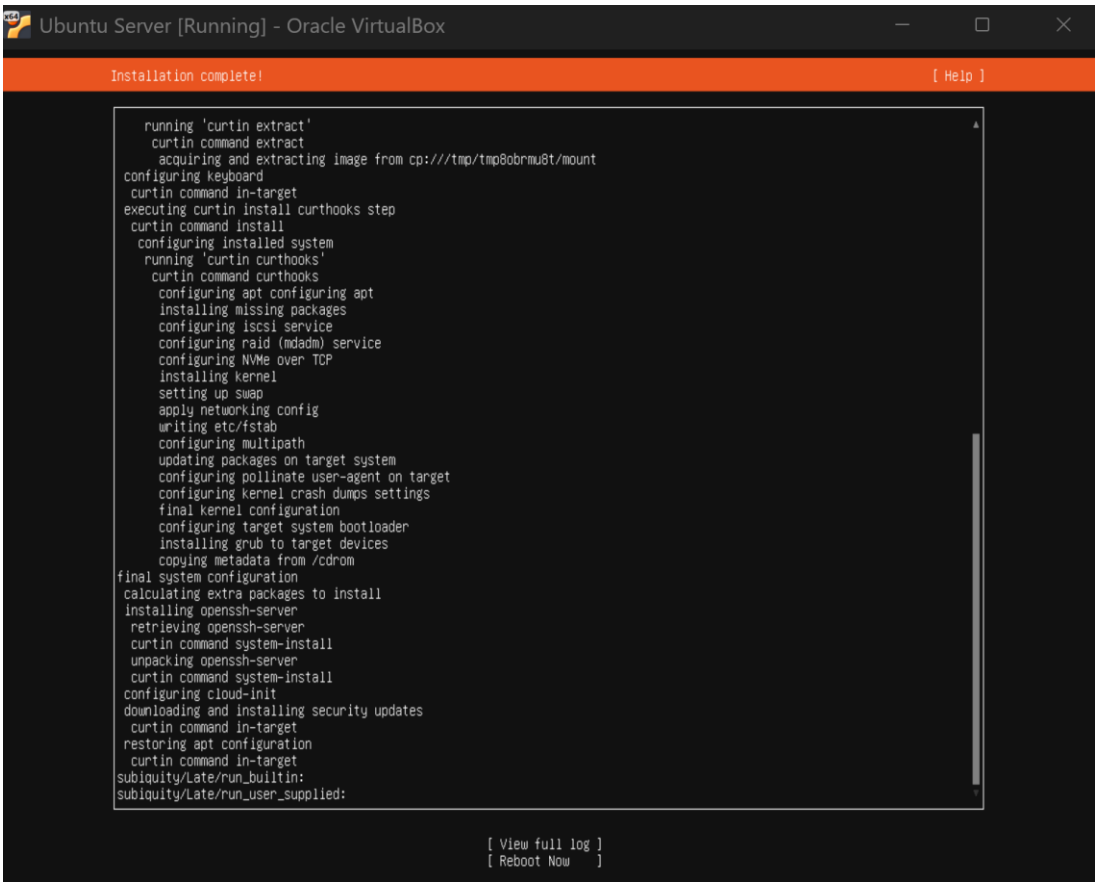


Figure 12: the "Installation complete" screen with "Reboot Now"

Summary of the installation choices:

- **Ubuntu version** : Ubuntu Server 26.04.1 LTS.
- **Installer language** : English
- **Keyboard** : French (AZERTY)
- **Network** : DHCP (NAT).
- **Proxy** : None
- **Mirror** : Default
- **Storage** : Entire 60 GB virtual disk
- **Server name** : ubuntu-server.
- **Ubuntu Pro** : Skipped
- **OpenSSH server** : Installed
- **Extra snaps** : None

References :

[1] A. Ahmadi, TP 1: Mise en place de l'environnement de virtualisation, Cloud Computing et Virtualisation (Partie C), ENSAM Meknès.

[2] Canonical, Ubuntu Server documentation

5. First steps in Ubuntu and network modes

Now that Ubuntu Server is installed, we can login and check that the virtual machine sees the resources we gave it. Ubuntu Server has no graphical interface, so everything is done by typing commands in the console.

5.1 Logging in

At the login prompt, type the username created during the installation and press **Enter**, then type the password and press **Enter** again. For security reasons, nothing appears on screen while you type the password, not even asterisks, so this is normal.

Once logged in, you see a welcome message and the command prompt, which means the system is ready.

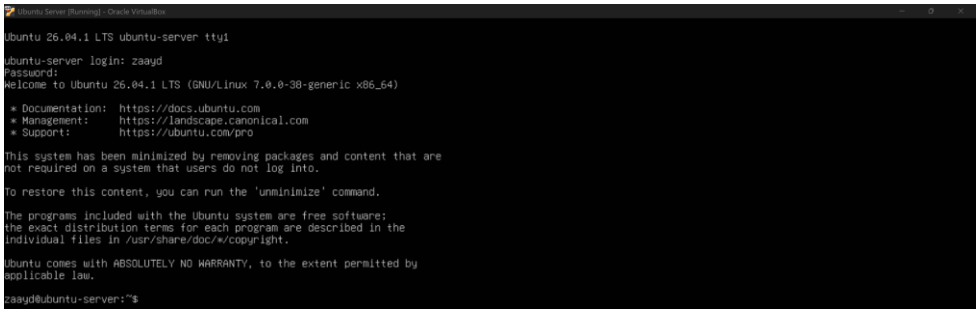


Figure 13: the login screen and the welcome message after a successful login

5.2 Changing the console font (optional)

By default, the text in the console can be hard to read on a high-resolution screen. To make it larger, run

```
sudo dpkg-reconfigure console-setup
```

The system asks for your password, because **sudo** runs the command with administrator rights. A text-based wizard then opens. Use the arrow keys to move and **Enter** to validate:

- **Encoding** : keep the default (UTF-8)
- **Character set:** Arabic
- **Font** : DejaVu
- **Font Size** : choose a larger size, for example **16x32** (very large) or **12x24** (medium).

The console updates as soon as you validate



Figure 14: the console after updating the font, showing the larger text

5.3 Checking the system

The first thing to verify is that the virtual machine really sees the resources we configured in VirtualBox. We use a few basic commands for this

5.3.1 Server name and version

```
hostname
cat /etc/os-release | grep VERSION
```

The first command shows the server name chosen during the installation. The second shows the system details:

The **VERSION_ID** and **VERSION** lines give the exact Ubuntu version. Which should be 26.04.1 LTS

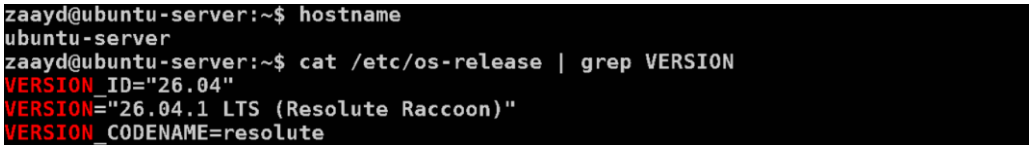


Figure 15: output of hostname and cat /etc/os-release | grep VERSION

5.3.2 Memory and processor

```
free -h
lscpu | grep "CPU(s):"
```

free -h shows the memory presented to the virtual machine, in a human-readable format. The total should be close to 2 GB we allocated.

Lscpu describes the processor, and the **CPU(s)** line should show 2, matching the two virtual processors set in VirtualBox, even though the host has 20 logical processors.

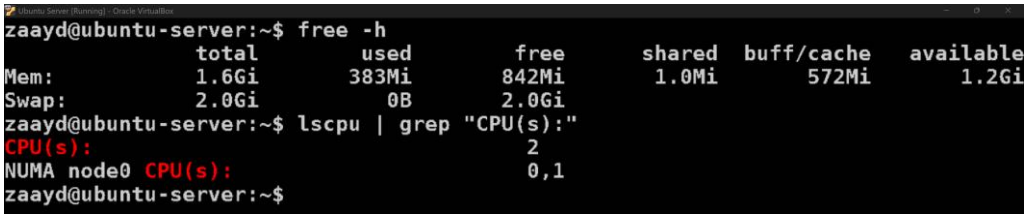


Figure 16: output of free -h and lscpu | grep "CPU(s):" showing 1.6 GiB of memory and 2 CPUs

5.3.3 Disk and file systems

```
lsblk
df -h
```

lsblk lists the disks and their partitions. The virtual disk appears with a size of about 60 GB.

df -h shows the file systems with the space used and available, which tells us how much room is left on the system

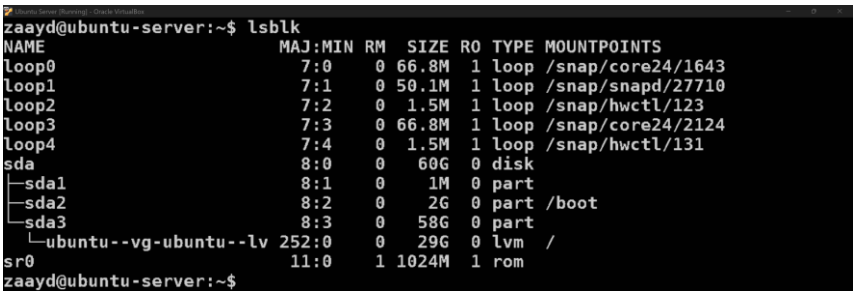


Figure 17: output of lsblk

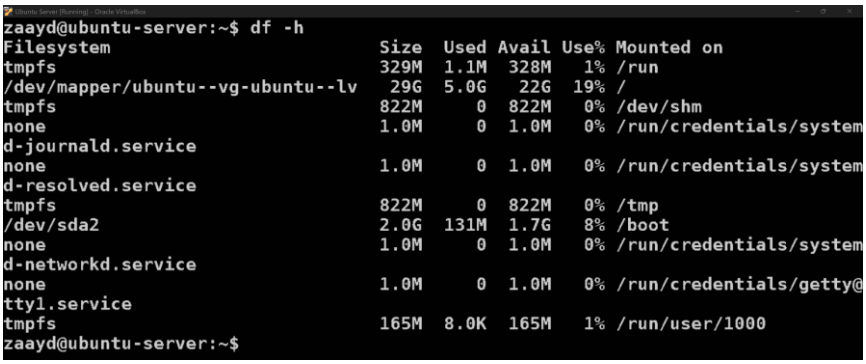


Figure 18: output of df -h

5.3.4 Summary of the commands :

To sum up, the list below shows the commands used in this section to inspect the virtual machine. Each line follows the same format: the command first, then the information it displays.

- **hostname** : Name of the server
- **cat /etc/os-release** : Ubuntu version and edition
- **free -h** : Memory seen by the VM
- **lscpu** : Processor, including the number of CPUs
- **lsblk** : Disks and partitions
- **df -h** : Used and available disk space

5.4 Checking the network

The virtual machine is attached to the NAT network, so VirtualBox should provide it with an IP address and internet access automatically. We check this with four commands.

5.4.1 IP address and routing

```
ip addr
ip route
```

ip addr lists the network interfaces and their IP addresses. Besides the loopback interface **lo**, we should see the main interface (named **enp0s3** or similar) with an IPv4 address given by DHCP. In NAT mode, VirtualBox usually assigns an address in the **10.0.2.x** range.

Ip route shows the routing table, and the line starting with **default** indicates the gateway used to reach other networks, which is the route toward the internet.

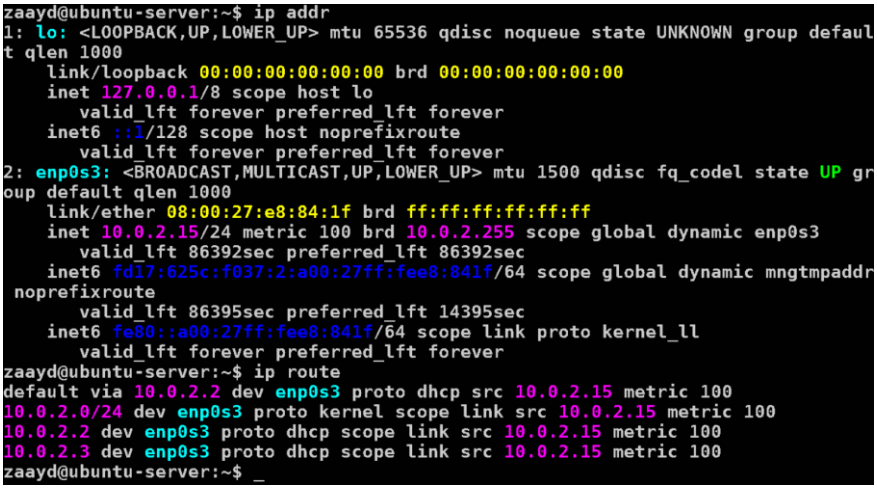


Figure 19: output of ip addr and ip route, showing the interface, its IP address, and the default route

5.4.2 Connectivity test

```
ping -c 4 8.8.8.8
ping -c 4 google.com
```

The first **ping** sends four packets to a public IP address, which checks that the virtual machine can reach the internet. The second one uses a domain name, so it also checks that DNS resolution works. If the first test succeeds and the second fails, the problem is the DNS configuration rather than the connection itself.

```
zaayd@ubuntu-server:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=26.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=46.2 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=111 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=26.2 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 26.153/52.624/111.489/34.936 ms
zaayd@ubuntu-server:~$ ping -c 4 google.com
PING google.com (172.217.20.238) 56(84) bytes of data.
64 bytes from lcmada-ah-in-f14.1e100.net (172.217.20.238): icmp_seq=1 ttl=255 ti
me=30.5 ms
64 bytes from lcmada-ah-in-f14.1e100.net (172.217.20.238): icmp_seq=2 ttl=255 ti
me=29.2 ms
64 bytes from lcmada-ah-in-f14.1e100.net (172.217.20.238): icmp_seq=3 ttl=255 ti
me=29.2 ms
64 bytes from lcmada-ah-in-f14.1e100.net (172.217.20.238): icmp_seq=4 ttl=255 ti
me=29.0 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 29.015/29.501/30.529/0.599 ms
zaayd@ubuntu-server:~$ _
```

Figure 20: output of two ping commands with 0% packet loss

5.5 Checking SSH and SSHD services

During the installation, we chose to install the OpenSSH server. The two sides of SSH should not be confused:

- **ssh** : is the client used to connect to a remote machine
- **sshd :** is the server (the daemon) that accepts connections.

To check if the service is running :

```
systemctl status ssh
```

Two lines of the output matter:

- **Active** : the mention **active (running)** means the SSH service is working
- **Loaded :** the mention **enabled** means the service starts automatically each time the system boots.

If the service is not enabled, activate it with:

```
sudo systemctl enable ssh
```

To leave the status screen and go back to the prompt press **q**.

```
zaayd@ubuntu-server:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: e
nabled)
     Active: active (running) since Sun 2026-10-18 02:47:33 UTC; 14s
ago
 Invocation: eb9b7ad6576e4e4286fc8515df2c0856
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 895 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 920 (sshd)
      Tasks: 1 (limit: 1920)
     Memory: 1.6M (peak: 2.3M)
        CPU: 113ms
     CGroup: /system.slice/ssh.service
             └─920 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 s
tartups"

Oct 18 02:47:33 ubuntu-server systemd[1]: Starting ssh.service - OpenBSD Secure
Shell server...
Oct 18 02:47:33 ubuntu-server sshd[920]: Server listening on 0.0.0.0 port 22.
Oct 18 02:47:33 ubuntu-server sshd[920]: Server listening on :: port 22.
Oct 18 02:47:33 ubuntu-server systemd[1]: Started ssh.service - OpenBSD Secure S
hell server.
```

Figure 21: output of systemctl status ssh showing "active (running)" and "enabled"

5.6 VirtualBox network modes

So far, our virtual machine uses **NAT**, the default mode. In this mode, the machine can reach the internet, but Windows cannot start a connection to it, so an SSH connection from Windows fails by default. VirtualBox offers other network modes, which we will explore in the next lab. For now, we need only two ways to reach the server from Windows.

5.6.1 Solution 1: Bridged Adapter

In Bridged mode, the virtual machine connects directly to the physical network and gets its own IP address, like a real computer.

1. Shut down the virtual machine completely **sudo systemctl poweroff**.

2. In VirtualBox, select **Ubuntu Server**, then open **Settings -> Network**.

3. In **Attached to**, change **NAT** to **Bridged Adapter**, and make sure the **Name** field shows your active network card.

4. Click **OK** and start the virtual machine again.

5. Login and run **ip a** to get the new IP address. It now belongs to the same network as Windows (for example: 192.168.x.x)

6. On Windows, open command prompt (CMD) and connect with:

```
ssh zaayd@192.168.1.13
```

On the first connection, SSH asks you to confirm the server's fingerprint. Type **yes**, then enter the password.

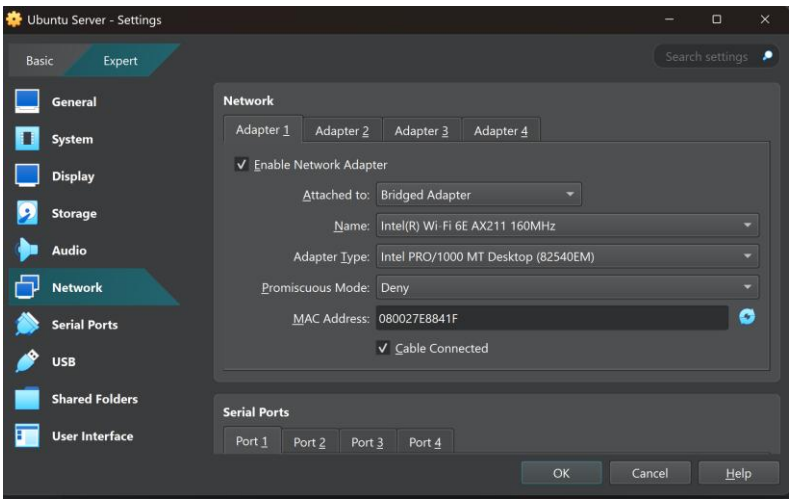


Figure 22: Settings -> Network with "Bridged Adapter" selected

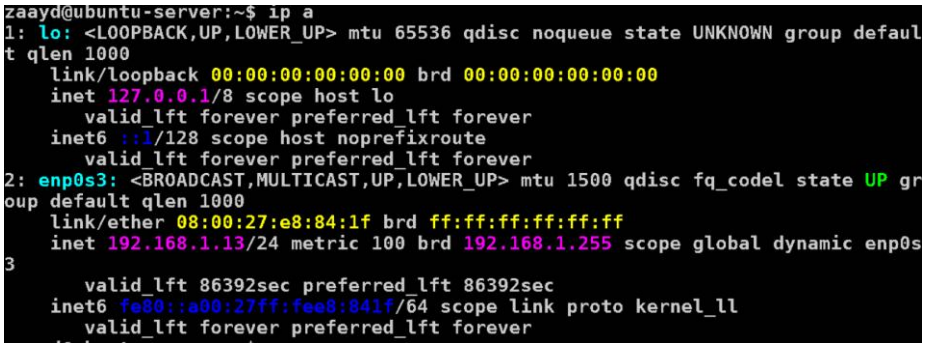


Figure 23: the output of ip a showing the new IP address

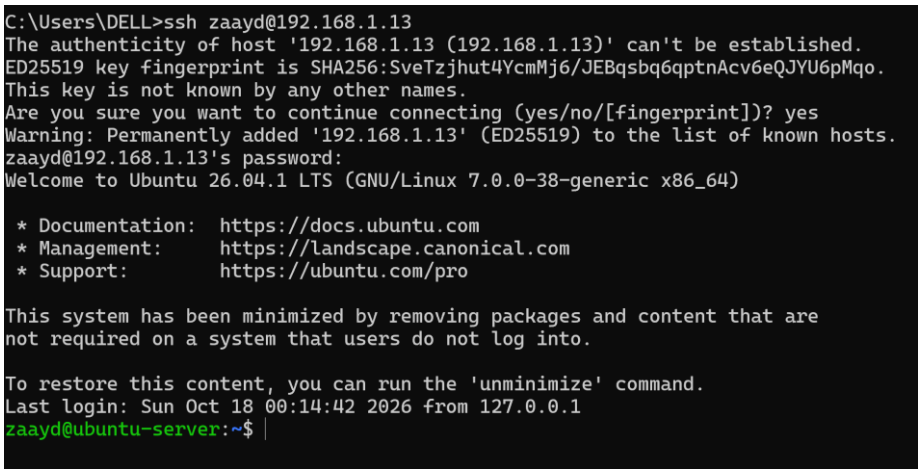


Figure 24: the SSH connection from the Windows command prompt

5.6.2 Solution 2: Port forwarding (staying in NAT)

If bridged mode is not possible, for example on a school or public network that restricts new devices, we can keep NAT and ask VirtualBox to redirect a port of Windows to the SSH port (22) of the virtual machine.

1. Shut down the virtual machine

2. Open **Settings -> Network**, expand **Advanced**, and click **Port Forwarding**

3. Add a rule with the green + button :

- **Name** : SSH
- **Protocol** : TCP
- **Host IP** : 127.0.0.1
- **Host Port** : 2222
- **Guest IP** : Leave empty
- **Guest Port** : 22

4. Click **OK** and start the virtual machine.

5. On Windows, connect through the local port:

```
ssh zaayd@127.0.0.1 -p 2222
```

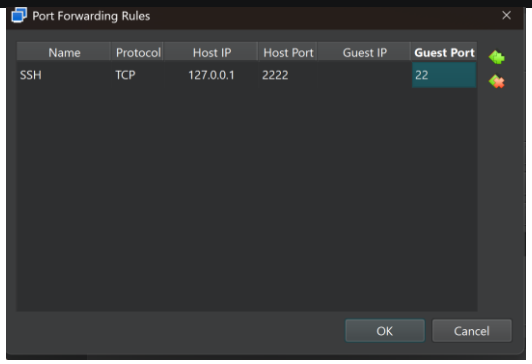


Figure 25: the port forwarding rule in VirtualBox

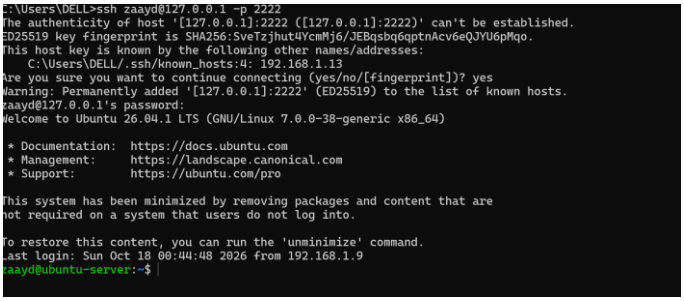


Figure 26: the SSH connection using port 2222

5.6.3 Conclusion

Both solutions give the same result: a remote terminal on the server from Windows. Bridged mode is simpler, while port forwarding works on any network.

For now, we connected with the default SSH settings, which is enough for a lab environment. A server exposed on a real network needs a more careful configuration of the SSH daemon (**sshd**), such as stronger authentication and restricted access. We will come back to this in a later lab, where we will configure **sshd** securely.

6. Snapshots and VBoxManage

Our virtual machine is now installed and working.

Before changing anything else, we save this clean state with a snapshot, then we discover VirtualBox's command-line tool.

6.1 Shutting down the virtual machine cleanly

A snapshot is best taken when the machine is powered off, because the disk is then in a stable state. We shut down the system properly from the console with:

```
sudo systemctl poweroff
```

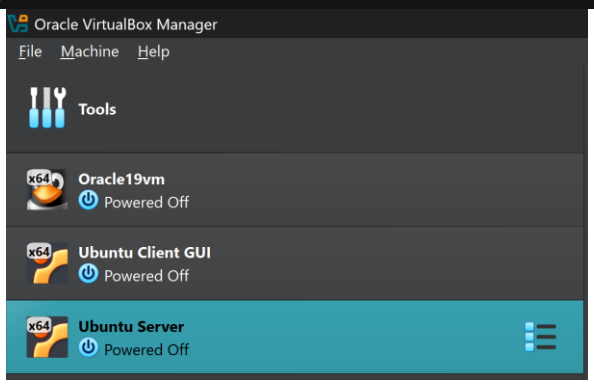


Figure 27: the VM showing "Powered Off" in VirtualBox Manager

6.2 Creating the reference snapshot

With the virtual machine powered off, we take a snapshot of this clean installation:

1. In the VirtualBox Manager, select **Ubuntu Server**.

2. Open the **Snapshots** tab (in the toolbar next to the machine name) and click **Take**.

3. Enter the name **Clean-Install**, and optionally a description such as "Fresh Ubuntu Server 26.04.1 LTS with OpenSSH".

4. Click **OK**.

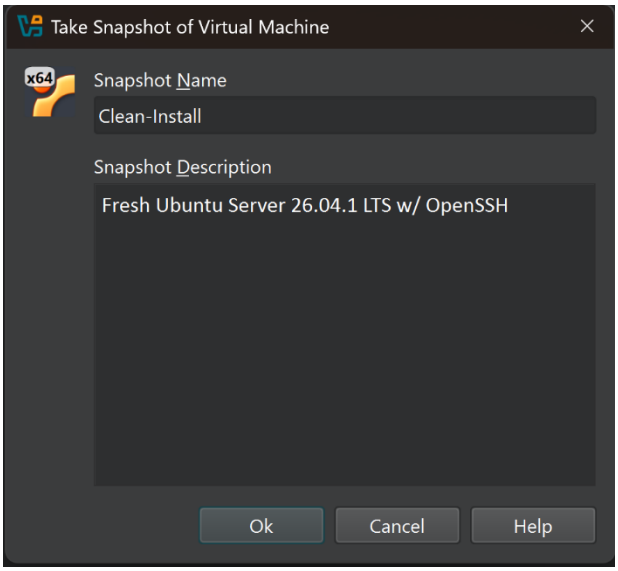


Figure 28: the "Take Snapshot" window with the name Clean-Install

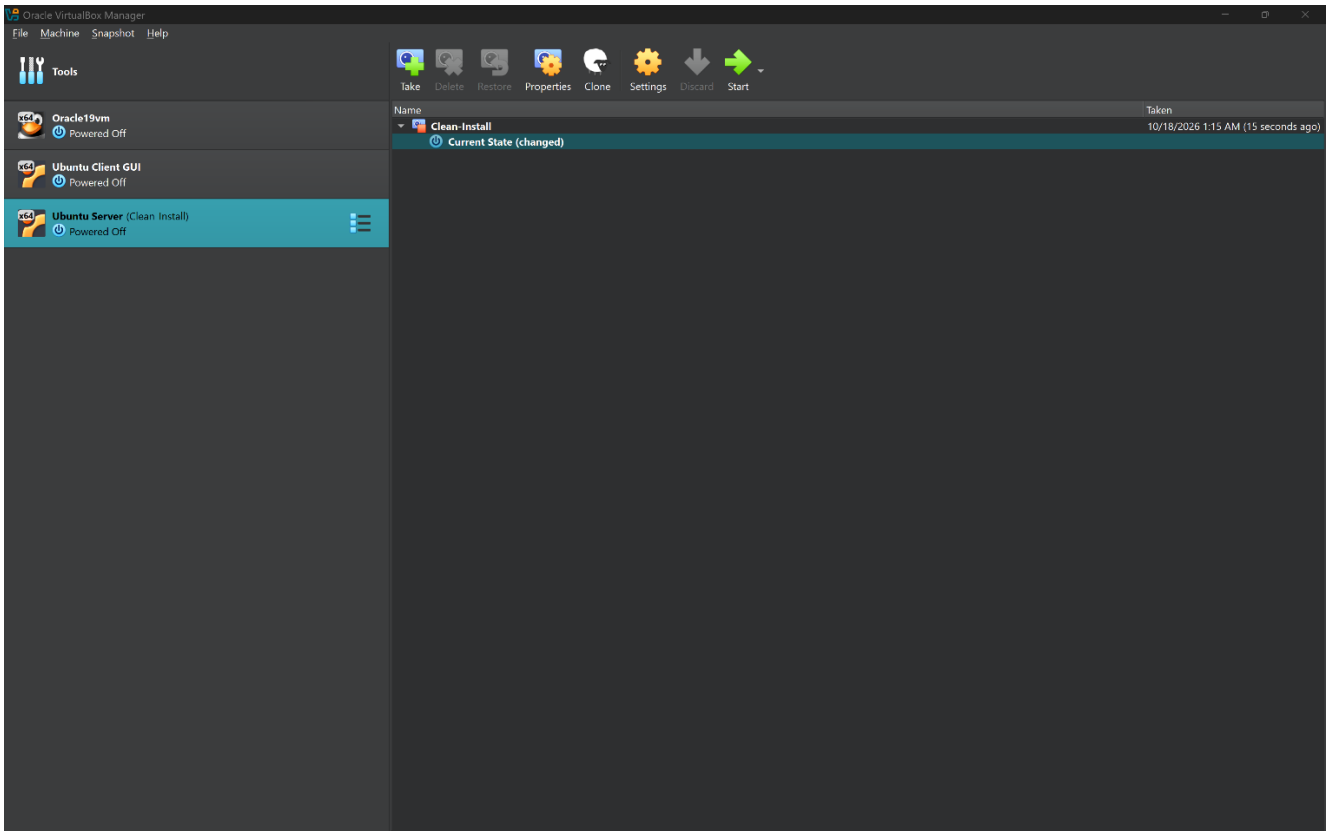


Figure 29: the Snapshots tab showing Clean-Install and Current State

A snapshot records the state of the virtual machine at a precise moment. If we later break the system, for example by deleting an important file or misconfiguring the network, we can go back to this clean state in a few seconds instead of reinstalling Ubuntu.

6.3 Snapshot or Backup

A snapshot is useful, but it is not a backup. The two are often confused, so it is worth understanding the difference.

When we take a snapshot, VirtualBox freezes the current virtual disk and records every later change in a separate file. The snapshot therefore depends on the original disk file. If that file is lost or corrupted, or if the physical drive fails, the snapshot is lost with it, because everything sits on the same drive.

A backup, on the other hand, is an independent and complete copy of the virtual machine, ideally stored somewhere else, such as an external drive or another computer. It survives even if the original machine or drive is destroyed.

6.4 Discovering VBoxManage

VirtualBox can be controlled without its graphical interface, using the command-line tool **VBoxManage**. It is very useful for administrators, because commands can be repeated, scripted, and used on machines without a screen. We run it from **PowerShell** on Windows, not inside Ubuntu.

6.4.1 Adding VBoxManage to the PATH

On Windows, the VirtualBox folder is not always in the system **PATH**, the list of folders where Windows looks for commands. If PowerShell says that : **VBoxManage is not recognized**, we add the folder ourselves:

1. Open the Start menu, search for **Edit the system environment variables**, and open it.

2. In the **System Properties** window, click **Environment Variables**.

3. In the **System variables** section, select **Path** and click **Edit** (If you don't have administrator rights, use the **Path** variable in the **User variables** section instead).

4. Click **New** and type the VirtualBox folder the default folder is "**C:\Program Files\Oracle\VirtualBox**".

5. Click **Ok** in all three windows to save.

6. Close PowerShell and open it again, because an already open window doesn't see the change.

7. Check that it works:

```
VBoxManage --version
```

If a version number appears, the PATH is configured correctly.

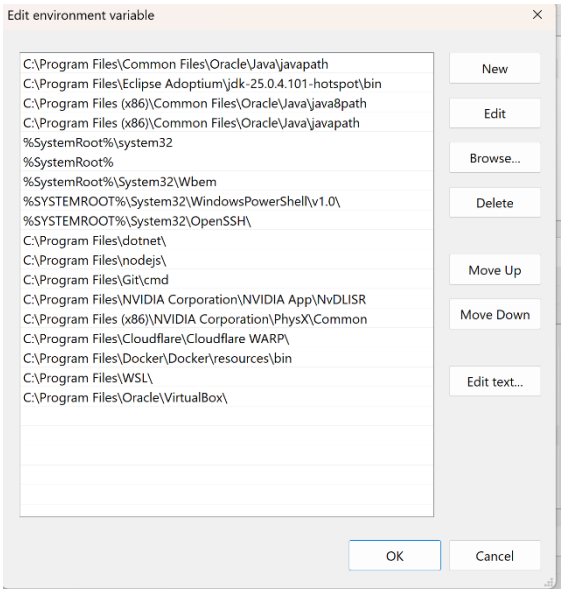


Figure 30: the Environment Variables window with the VirtualBox folder added to Path

6.5 Listing the virtual machines

```
VBoxManage list vms
```

This command lists all the registered virtual machines with their name and unique identifier (UUID). Our machine, **Ubuntu Server**, should appear in the list.

```
PS C:\Windows\system32> VBoxManage list vms
"Oracle19vm" {92c76db5-eba6-4e83-aa08-a8f5f5fb51a6}
"Ubuntu Client GUI" {eb6ab430-7001-4a2a-83e5-fc9c7b52f080}
"Ubuntu Server" {50ccfa55-fb4d-4cdd-895d-1df4107f5aaa}
```

Figure 31: output of VBoxManage list vms in PowerShell

6.6 Showing the details of a machine

```
VBoxManage showvminfo "Ubuntu Server"
```

The name is written between quotes because it contains a space. The output is long, so we can filter the lines that interest us:

```
VBoxManage showvminfo "Ubuntu Server" | Select-String "Memory size|Number of CPUs|NIC 1:"
```

We find the same information as in the graphical interface: the memory (2048 MB), the number of processors (2), and the network adapter with its mode. Both tools read the same configuration, so the values must match what we set in **Settings**. The graphical interface is easier for a first approach, while VBoxManage is more complete and suited to automation.

```
PS C:\Windows\system32> VBoxManage showvminfo "Ubuntu Server" | Select-String "Memory size|Number of CPUs|NIC 1:"

Memory size:                 2048MB
Number of CPUs:              2
NIC 1:                       MAC: 080027E8841F, Attachment: NAT, Cable connected: on, Trace: off (file: none), Type:
82540EM, Reported speed: 0 Mbps, Boot priority: 0, Promisc Policy: deny, Bandwidth group: none
```

Figure 32: output of VBoxManage showvminfo, filtered on memory, CPUs, and network

7. Cloning the virtual machine

Once a virtual machine is installed and configured, we can duplicate it instead of installing Ubuntu again. This is how administrators deploy several servers quickly, and we will reuse the clone in the next labs.

7.1 Creating the clone

Shut down the virtual machine first, then right-click **Ubuntu Server** in the VirtualBox Manager and choose **Clone**. In the wizard, set:

- **Name:** Ubuntu Client #1
- **Path:** the folder where the new machine files will be stored
- **Clone Type:** **Full Clone**
- **Snapshots:** **Current Machine State**.
- **MAC Address Policy:** **Generate new MAC addresses for all network adapters**.

Then click **Finish**.

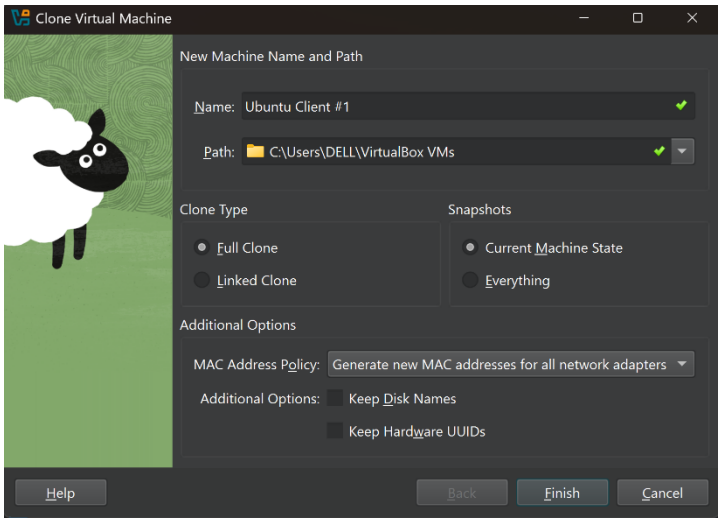


Figure 33: the Clone Virtual Machine window with these options selected

Each option has a reason:

1. A **full clone** copies the whole virtual disk, so the new machine is completely independent. A linked clone shares the disk of the original and stays dependent on it, which saves space but breaks if the original is deleted.

2. Current Machine State copies the machine as it is now, without the snapshot history.

3. New MAC addresses are essential. Two machines with the same MAC address on the same network cause conflicts, because the address is supposed to be unique

7.2 Checking the clone

When the cloning is done, **Ubuntu Client #1** appears in the list of machines, next to the original. It has the same resources (2 GB of RAM, 2 vCPUs, 60 GB disk) and the same installed system, including the same user account and password.

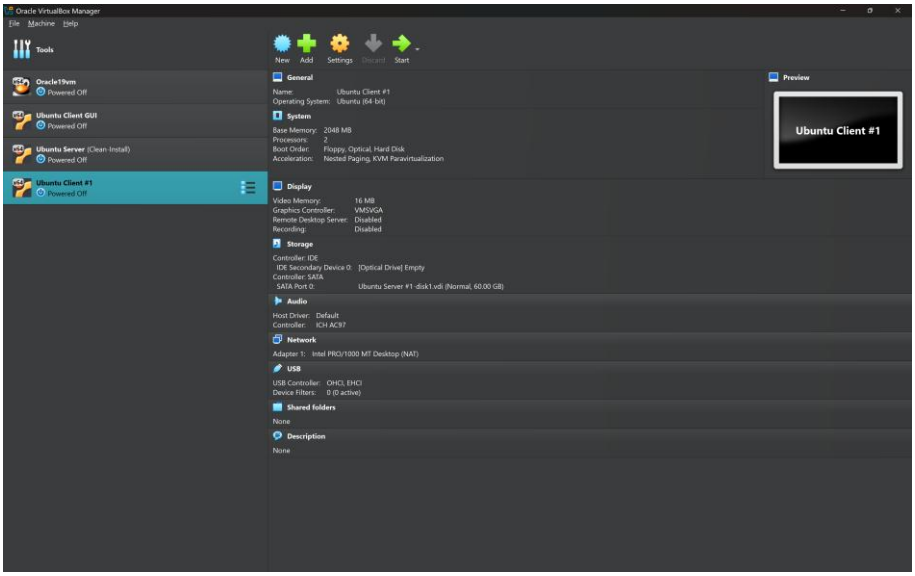


Figure 34: the VirtualBox Manager showing both machines, Ubuntu Server and Ubuntu Client #1

Conclusion

In this session, we turned a Windows PC into a small virtualization platform. We started by understanding the three roles involved: the host machine, the hypervisor (VirtualBox), and the guest machine (Ubuntu Server). We checked the resources of the host, created a virtual machine with a reasoned allocation of memory, processors, and disk, and installed Ubuntu Server 26.04.1 LTS step by step.

Inside the virtual machine, the commands we ran showed that it only sees the resources we gave it, not those of the physical host: 2 virtual processors and about 1.6 GiB of memory, while the host has 20 logical processors and 64 GB of RAM. This is the central idea of virtualization, where virtual resources are carved out of physical ones.

We then connected to the server from Windows through SSH, using Bridged mode or port forwarding, and saw why the default NAT mode is not enough for this. Finally, we created a snapshot of the clean installation, understood that it is not a replacement for a backup, discovered VBoxManage, which gives access to the same settings from the command line, and cloned the machine to prepare for the next labs.

Along the way, a few things did not work on the first try, and that is part of learning. As the saying goes:

"Failure isn't fatal, it's just an opportunity to do better"

This lab is only the foundation. In the next labs, we will explore the other VirtualBox network modes and explore the core tasks of Linux administration: managing users and groups, controlling permissions, working with services…